

Assignment 2

Bino Matteo 2158293	matteo.bino@studenti.unipd.it
Caregnato Gianluca 2157859	gianluca.caregnato@studenti.unipd.it
Meneghetti Federico 2157857	federico.meneghetti.2@studenti.unipd.it
Github:	group_number_4_assignment_2
Video:	video assignment 2

1 Introduction

This report describes the software architecture and implementation for the second assignment of the Intelligent Robotics course. The objective of the system is to robustly detect, identify, and swap the positions of two cubes using a robotic manipulator. The solution is divided into 3 main packages: **cubes_info** developed by Federico Meneghetti, **motion_controller** developed by Gianluca Caregnato and **logic_controller** developed by Matteo Bino.

1.1 Cubes Info Package

1.1.1 Pose detection

To identify the AprilTags, we developed a launch file that creates a *node container* with *two composable nodes*:

- **Image_proc::RectifyNode**, which allows image rectification (corrects distortion).
- **Apriltag_ros::AprilTagNode**, which enables AprilTag detection using the rectified image. The AprilTagNode is configured via the **tags_36h11.yaml** file, which specifies the tag family (36h11), the size (5 cm), the Tag IDs to search for (1 and 10).

The node **find_cubes_position_node** subscribes to the **apriltag/detections** topic, which allows it to automatically obtain the tag frame IDs and, using the transformations available in **/tf**, it then computes the pose of Tag 1 and Tag 10 with respect to **base_link**. The two poses are then inserted into a **cubes_info::msg::CubesPoses** message, which contains: header, the array of cube IDs and the array of cube poses. Then the message is published to the **/cubes_poses** topic.

1.1.2 Color detection

To extract the color of the cubes, the node **detect_color_node** was created, which implements an action server. It receives as input the cube's pose expressed with respect to the **base_link** frame and returns the cube's color as a string.

Upon receiving a goal, the node transforms the cube's pose from the **base_link** frame to the camera frame (**external_camera/link/rgb_camera**) and, using the intrinsic camera parameters provided by the **/rgb_camera/camera_info** topic, projects the pose onto the image plane. This way, the corresponding point is obtained in pixel coordinates, from which a small region of interest (ROI) is constructed, slightly translated downward, which contains the colored portion of the cube. More precisely, the extracted area is converted to the HSV color space and, using thresholds of the H (Hue) channel value, the cube's color is obtained. In practice, to obtain the cube position in pixel coordinates, the following formula is used:

$$\tilde{\mathbf{m}} = \mathbf{P} \tilde{\mathbf{X}} = \mathbf{K} \begin{bmatrix} \mathbf{I} & \mathbf{0} \end{bmatrix} \mathbf{T}_{cw} \tilde{\mathbf{X}}_w$$
$$\tilde{\mathbf{m}} = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_u & 0 & u_0 & 0 \\ 0 & f_v & v_0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} \quad \text{with} \quad \mathbf{K} = \begin{bmatrix} f_u & 0 & u_0 \\ 0 & f_v & v_0 \\ 0 & 0 & 1 \end{bmatrix}$$

The intrinsic parameters f_u and f_v are respectively the horizontal and vertical focal lengths in pixels. The values u_0 , v_0 represent the image center in pixel coordinates. The vector $\tilde{\mathbf{X}}_w$ represents the cube homogeneous coordinates expressed in the world frame (in this case, the **base_link** frame), and $\mathbf{T}_{cw}$ is the homogeneous transformation matrix that converts points from the world (**base_link**) frame to the camera frame.

1.2 Motion Controller Package

1.2.1 RobotManipulator

The **RobotManipulator** class acts as the bridge between the high-level decision logic and the simulated robot, utilizing the MoveIt planning framework. The class implements three action servers: **go_home** to return the robot to a safe configuration, **move_cube** to handle complex pick-and-place sequences and **reset** to return the robot in the starting position. A key challenge in this system is the potential noise in the AprilTag pose estimation. To mitigate this, the pick-and-place operations implement a coordinate approximation strategy.

This approximation effectively snaps the noisy sensor data to a clean grid, ensuring the gripper aligns perfectly with the standard cube geometry. The manipulation logic is designed for safety and robustness through a multi-stage process:

1. **Approach:** The arm moves to a pre-defined position directly above the target. This vertical alignment is crucial to prevent lateral collisions with the target cube itself or other objects residing in the workspace.
2. **Pick:** The end-effector descends to the specific grasp height. This is a delicate operation executed via a Cartesian path planning constraint. This ensures the gripper moves perfectly vertically to the object, effectively forking it and preventing any lateral drift that might knock the cube over. This pick position incorporates a Z-axis offset based on the gripper's dimensions: the goal is to position the fingers low enough to secure the object, but high enough to avoid driving the object into the table surface. Once aligned, the gripper closes to capture the cube.
3. **Depart:** After grasping, the arm raises the object vertically to ensure it does not drag across the table surface. Ideally, the arm would lift high enough to clear all surrounding obstacles. However, due to kinematic constraints (the arm operating near its maximum extension), the vertical lift is limited. Consequently, we must rely on strategic placement of other objects to ensure a collision-free path during the transfer.
4. **Place procedure:** The placement sequence mirrors the geometry of the pick strategy. The arm executes the same vertical Approach and Depart motions. However, the central step is inverted: the pick position becomes the place position, where the arm lowers the attached object to the surface and the gripper opens to release it.

1.3 Logic Controller Package

The Logic Controller is responsible for high-level decision-making. By adopting a Behavior Tree (BT) architecture, the system is modular allowing the robot to handle complex task sequences, such as the cube-swapping routine, while maintaining clear recovery paths in case of failures.

1.3.1 Behavior Tree paradigm, Data input/output handling

The core of the controller is a hierarchical tree structure that governs the mission lifecycle. This approach separates *what* the robot should do (the logic) from *how* it is physically achieved (the implementation).

The routine is organized into sequences that must succeed in order. To enhance robustness, the controller utilizes *Decorators* and *Fallbacks*. For instance, if a motion command fails due to a planning error, a fallback branch is triggered to safely reset the robot to a home configuration rather than allowing the system to stall in an uncertain state.

Data persistence across the routine is managed via a centralized "Blackboard." This acts as a shared memory space where perception nodes can write environmental data (such as detected cube coordinates) and execution nodes can read those values to parameterize their motion goals.

To bridge the gap between human-readable logic and ROS 2 message types, the controller implements custom parsing logic. This allows complex spatial data, such as 3D poses and orientations, to be defined directly within the high-level mission description as serialized strings, which are then automatically converted into geometric messages at runtime.

1.3.2 Mapping Behavior Tree to communication methods (topics/actions)

The transition from a BT action to a physical movement in the simulation is handled by specialized action wrappers. These wrappers bridge the synchronous nature of the tree with the asynchronous nature of ROS 2.

- **State-Aware action wrappers:** Most behaviors are implemented as state-aware entities. When a BT node (robot state) is entered, it initializes a ROS 2 Action Client and sends a goal. It then remains in a "Running" state, yielding control back to the tree while waiting for the robot to complete its physical movement. This non-blocking execution is essential for maintaining system responsiveness.
- **Synchronous perception monitoring:** The localization phase employs a topic-monitoring strategy. The controller subscribes to environmental updates and implements a "settling logic." It deliberately delays the transition to the next task until the incoming data stream confirms that the target objects have reached a stable, stationary state in the simulation, preventing the robot from attempting to grasp moving targets.

A summary of the high-level robot procedures can be found in Table 1.

2 System Flow

We defined as **cube 1** the red cube, that is the nearest to the camera, and as **cube 2** the blue cube, that is the furthest. Clearly, initially our system is unaware of the colors of the cubes, they are determined at runtime. The overall system behavior is governed by the sequence defined in the Behavior Tree (`swap_cubes.xml`). The flow proceeds as follows:

1. **GetCubesPoses (Localization):** The robot attempts to localize the cubes. This is wrapped in a `RetryUntilSuccessful` decorator, ensuring the system waits until it has a valid fix on the cube positions before proceeding.
2. **Go Home (Initialization):** The robot moves to the home position to ensure a safe, collision-free starting configuration.
3. **GetCubeColor (Color Verification):** The system invokes the perception node to detect and log the colors of both cube 1 and cube 2 at their respective locations.
4. **Swap Sequence:** The core logic performs the swap using a temporary buffer position:
 - (a) **MoveCube (1)** from its initial position and place it at a temporary buffer location: ($x = 0.5$, $y = -0.1$, $z = 0.4$, $rx = 1.0$, $ry = 0.0$, $rz = 0.0$, $rw = 0.0$).
 - (b) **MoveCube (2)** from its initial position and place it at cube 1's previous location.
 - (c) **MoveCube (1)** from ($x = 0.5$, $y = -0.1$, $z = 0.4$, $rx = 1.0$, $ry = 0.0$, $rz = 0.0$, $rw = 0.0$) and place it at cube 2's previous location.
5. **Recovery Fallback:** The swap sequence is wrapped in a `Fallback` strategy. If any step of the swap fails (e.g., motion planning failure), the `RecoveryLogic` sequence is triggered, which commands the robot to go home before terminating with failure, effectively resetting the system state for safety.
6. **GetCubeColor (Color Verification):** The system invokes again the perception node to detect and log the colors of both cube 1 and cube 2 at their respective locations, that now are switched. Now cube 2 is closer to the camera than before.
7. **GoHome:** The robot returns to the home position.
8. **Reset (Completion):** Finally, the robot returns to the initial position.

3 Discussion and Results

During the development of the system, we encountered several technical challenges that required specific architectural choices. One of the difficulties was the workspace limitation of the robotic arm. As noted in the Approach and Depart phase, when reaching for the furthest cube, the arm operates near its maximum extension. To overcome this problem, we had to carefully tune the approach and depart heights to find a trade-off between collision avoidance and kinematic feasibility.

Another problem was given by the AprilTags detection: the cubes were hard to detect when placed on the furthest table. Also, the detected cubes' positions were a little off the actual ones. This was probably due the fact that the view is not perpendicular to the AprilTags. As expected, using these coordinates as the targets to the motion planner resulted in erratic gripper behavior.

Since we thought that the robot could help with the color detection, we implemented an optional waypoint pose in the `move_cube` action server to function as a middle temporized stationary configuration of the robotic arm. This was supposed to move the cube closer and align it with the camera perspective, but it turned out that this feature was not necessary to recognize the color of the cubes. Also, this maneuver was not always exploitable since, to have the cube closer to the camera, the arm would reach its maximum extension and thus the planning would sometimes fail. However, the main trouble was that the cubes tended to slip out of the gripper grasp, probably due to simulation issues.

In conclusion, this project is able to correctly use the robotic arm to swap the cubes and detect their color. We combined **MoveIt** for the movement, **OpenCV** for the vision, and **Behavior Trees** for the logic. This helped us to create a modular architecture where the decision-making is separated from the low-level execution.

BT state	Communication type	Functional role
GetCubesPoses	Topic Subscriber	Populates blackboard with cubes poses.
GetCubeColor	Action Client	Identifies color at a specific pose.
MoveCube	Action Client	Executes multi-stage motion (pick and place).
GoHome	Action Client	Ensures the robot moves to a safe configuration.
Reset	Action Client	Ensures that the robot returns to the initial configuration.

Table 1: List of Behaviour Tree actions and corresponding description.

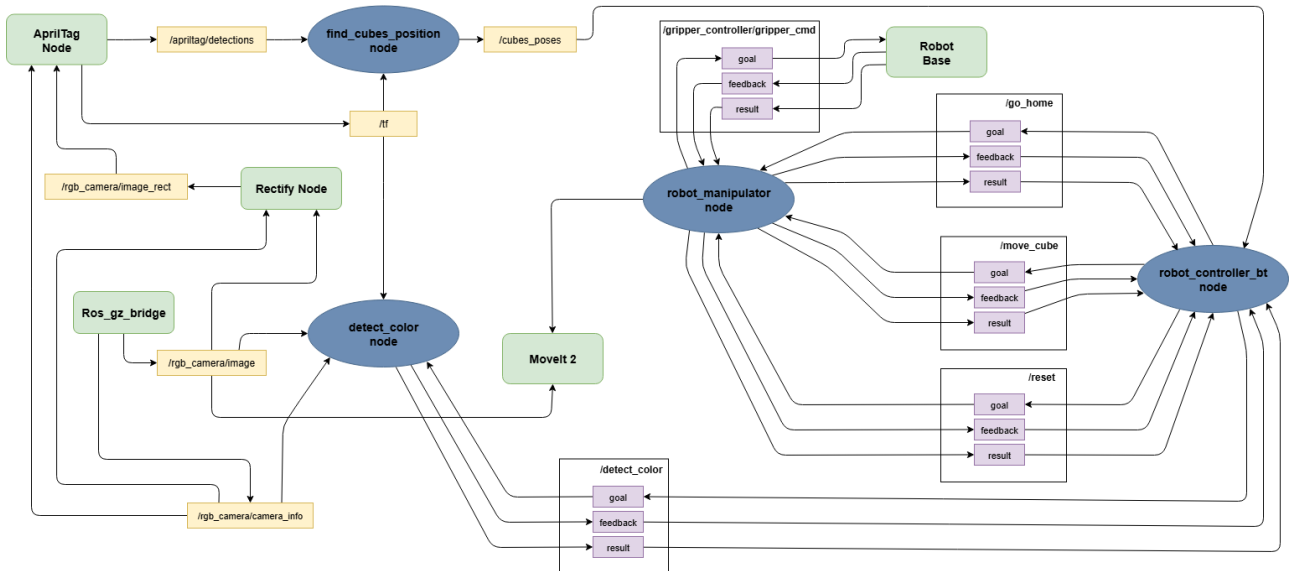


Figure 1: diagram of the most important nodes. Green nodes are provided by ir_launch, blue nodes are developed by us, orange labels are the topics, purple labels are the actions.